

# 从零开始的 SNN-WAM 具身智能实验计划书

面向 Spiking World-Action Adapters for Embodied Foundation Models

|      |                                                                                     |
|------|-------------------------------------------------------------------------------------|
| 定位   | 从不会具身实验开始，到能跑出第一组离线 benchmark、闭环仿真 success rate、SNN baseline 和 EGGROLL-style 后训练雏形。 |
| 主线   | LIBERO 数据 + 模型训练 + closed-loop rollout；之后再扩展到 ManiSkill、OpenVLA、Unitree。            |
| 第一目标 | 不要直接做完整 VLA/WAM 大模型，而是先做冻结视觉语言编码器 + SNN latent dynamics adapter。                    |
| 版本日期 | 2026-05-25                                                                          |

使用提醒：本计划书是可执行的项目手册，不是论文综述。软件版本和命令会变化，真正安装时以官方 README / docs 为准；本文件给出推荐顺序、检查点、实验对象和每一步应该产出的证据。

# 0. 一页总览：你到底要做什么

你要从零进入的不是“机器人硬件维修”，也不是“把 SNN 生硬塞进一个控制器”，而是一个更清晰的实验问题：在具身基础模型里，SNN 能否作为 world-action adapter，承担短期动态记忆、未来状态预测和动作调制？

| 层级           | 你要做的事                                                                |
|--------------|----------------------------------------------------------------------|
| 最小问题         | 给定当前图像、语言指令和历史动作，预测未来动作，并预测未来视觉 latent。                              |
| 为什么像 WAM     | 模型不是只做 observation -> action，而是同时学习 action-conditioned future state。 |
| 为什么用 SNN     | 让 SNN 承担时间动态、事件驱动、稀疏激活和短期状态记忆，而不是只替换一个 MLP。                          |
| 为什么需要仿真      | 离线预测误差不等于机器人能完成任务；必须把模型放回环境闭环执行。                                     |
| 为什么接 EGGROLL | 当目标变成 task success、动作平滑、安全约束和 spike 稀疏性时，很多指标不是普通可微 loss，适合黑箱或混合优化。  |

第一阶段的实验形态仍然和你熟悉的图像分类很像：数据集、dataloader、模型、训练、验证、测试、画图。真正的区别是第二阶段多了 closed-loop rollout：模型输出动作，环境执行动作，环境再返回下一帧观察。

```
图像分类: image -> model -> class -> accuracy
离线具身: image + instruction + action_history -> model -> future_action / future_latent -> MSE / cosine error
闭环具身: env.observe() -> model.action() -> env.step(action) -> next_observation -> success_rate
```

## 本计划的主结论

- 不要第一天就上 OpenVLA 7B、完整 WAM、真实宇树硬件或 EGGROLL；先跑通 LIBERO 的数据、baseline 和闭环 evaluation。
- 第一篇可汇报实验应当是：MLP / GRU / SNN temporal adapter 对比，加上 action prediction、future latent prediction、closed-loop success rate、spike rate 和鲁棒性曲线。
- EGGROLL-style ES 放在后训练阶段：先用 surrogate gradient 让 SNN 能学，再用 ES 优化不可微/系统级 fitness。

# 1. 从零建立概念：你需要知道的 20 个词

| 词                        | 本项目里的意思                                              |
|--------------------------|------------------------------------------------------|
| trajectory               | 机器人一次示范或一次执行的完整时间序列，例如图像、状态、动作、奖励、是否成功。              |
| observation              | 机器人在某一时刻能看到/感到的信息，可能包括 RGB 图像、深度、本体状态、关节角、末端位姿。      |
| action                   | 机器人要执行的控制量，可以是末端位姿增量、关关节目标、速度或离散动作 token。            |
| language instruction     | 任务文字，例如 “put the mug on the plate”。在 VLA 中语言是任务条件。   |
| behavior cloning         | 从专家示范中监督学习动作预测，类似 image -> label，只是 label 变成 action。 |
| rollout                  | 把策略放进环境里连续执行一段时间，观察它是否完成任务。                          |
| closed-loop              | 动作会改变环境，下一步输入依赖上一步动作结果；这是具身学习和静态分类最大的区别。             |
| VLA                      | Vision-Language-Action，输入视觉和语言，输出机器人动作。              |
| WAM                      | World-Action Model，除了输出动作，还要建模动作如何改变未来世界状态。          |
| latent                   | 编码器输出的低维/高维特征，不直接预测像素，而预测视觉特征。                       |
| future latent prediction | 预测未来几步图像经过视觉编码器后的特征，是最小 WAM 的核心。                     |
| temporal adapter         | 接在大编码器后面的时序模块，负责利用历史和状态记忆。                           |
| SNN                      | Spiking Neural Network，用离散 spike 和膜电位动态来表示时序计算。      |
| surrogate gradient       | 用近似梯度训练 spike 离散发放函数的常规方法。                           |
| spike rate               | 单位时间/单位神经元的发放比例，是 SNN 稀疏性和能耗 proxy。                  |
| fitness                  | 黑箱优化中的评价函数，可以包括任务成功、动作平滑、安全惩罚、spike 稀疏等。             |
| EGGROLL-style ES         | 低秩扰动的 evolution strategies 思路，用并行推理式评估优化参数。          |
| simulator                | 机器人仿真环境，负责根据 action 更新物理世界并返回 observation。           |
| benchmark                | 固定任务、数据划分、评价指标和 baseline，使不同方法可                      |

|             |                                 |
|-------------|---------------------------------|
|             | 比较。                             |
| sim-to-real | 从仿真迁移到真实机器人，第一阶段不做主线，只作为后续可能方向。 |

## 2. 你熟悉的图像分类流程，如何变成具身实验

### 2.1 相同的部分

| 图像分类项目                 | SNN-WAM 最小实验                                                              |
|------------------------|---------------------------------------------------------------------------|
| 下载 CIFAR/ImageNet 子集   | 下载 LIBERO demonstration 数据                                                |
| 写 Dataset / DataLoader | 写 trajectory window dataset                                               |
| 输入 image, 标签 class     | 输入 image + instruction + action history, 标签 future action + future latent |
| 训练 ResNet / ViT        | 训练 MLP / GRU / SNN adapter                                                |
| 报告 accuracy / loss     | 报告 action error / future latent error / spike rate                        |

### 2.2 不同的部分

| 差异   | 图像分类          | 具身实验                          |
|------|---------------|-------------------------------|
| 样本结构 | 单张图片独立        | 时间序列窗口，前后帧有关                  |
| 预测对象 | 类别标签          | 连续动作、动作序列、未来状态 latent         |
| 误差传播 | 预测错一张不影响下一张   | 动作错会改变下一帧，误差会滚雪球              |
| 最终评价 | test accuracy | closed-loop success rate 才是关键 |

### 2.3 第一段可以不机器人

第一阶段完全可以在服务器或工作站上完成。你需要的是：一个机器人数据集、一个仿真环境、一个训练脚本、一个闭环评估脚本。真实宇树 Go2/G1/H1 是第二阶段甚至第三阶段的事情。

### 2.4 最小工作流

- Step 0: 建 Python/Conda 环境，能 import torch。
- Step 1: 安装 LIBERO，下载 libero\_spatial 或 libero\_object 数据。
- Step 2: 跑官方 BC/RNN baseline，确认训练和 evaluation 能跑。
- Step 3: 自己写 trajectory\_window dataset，先训练 GRU adapter。
- Step 4: 加 future latent prediction，让模型变成最小 WAM adapter。
- Step 5: 把 GRU 换成 SNN，比较 MLP/GRU/SNN。
- Step 6: 进入 LIBERO 仿真 closed-loop evaluation，报告 success rate。
- Step 7: 加噪声、delay、frame drop、扰动，看 SNN 是否更稳。
- Step 8: 再考虑 EGGROLL-style ES 后训练。

### 3. 软件栈选择：先用什么，后用什么

软件栈不要一次铺太大。推荐主线是 LIBERO；备用和扩展是 LeRobot、ManiSkill、OpenVLA、Unitree RL Lab。

| 工具                              | 第一阶段是否需要 | 作用                                     | 选择理由                                    |
|---------------------------------|----------|----------------------------------------|-----------------------------------------|
| Python + PyTorch                | 必须       | 训练模型、写 dataloader、跑实验                  | 你已有机器学习经验，迁移成本最低。                       |
| VS Code + Git                   | 必须       | 代码编辑、实验管理、版本控制                         | 和你现有 workflow 一致。                       |
| LIBERO                          | 必须       | 语言条件机器人操作数据与仿真 benchmark               | 提供 130 个语言条件操作任务、官方训练和评估脚本，最适合入门。       |
| SpikingJelly / snnTorch / Norse | 第二步      | 实现 SNN adapter                         | SpikingJelly 是 PyTorch SNN 框架且中英文文档较完整。 |
| LeRobot                         | 可选       | 统一机器人数据、数据加载、后续真实数据管理                  | 降低机器人数据处理门槛，适合接入 HuggingFace 数据生态。      |
| ManiSkill3                      | 可选       | 高速 GPU 仿真、后续 rollout/ES 训练             | 适合大规模仿真评估和并行采样。                         |
| OpenVLA                         | 后续       | 真实 VLA backbone 或 fine-tuning baseline | 7B 开源 VLA，第一阶段不要直接端到端训练。                |
| Unitree RL Lab                  | 后续       | Go2/H1/G1 仿真和实机路径                      | 实验室有宇树硬件时可作为后续硬件验证线。                    |

#### 3.1 为什么主线选择 LIBERO

LIBERO 官方仓库说明它面向 multitask 和 lifelong robot learning，包含 130 个任务，分为 LIBERO-Spatial、LIBERO-Object、LIBERO-Goal 和 LIBERO-100 四个任务套件，并提供数据下载、训练和评估脚本。对你来说，它最大的好处是“有语言、有图像、有动作、有仿真 evaluation”，正好对应 VLA/WAM 最小实验。

#### 3.2 为什么暂时不直接上 OpenVLA

OpenVLA 是重要的后续目标，但它是 7B 级 VLA，工程门槛高。第一阶段你需要先理解机器人数据和闭环评估，而不是被大模型加载、显存、LoRA、动作 tokenizer 和评估脚本卡住。等 LIBERO 的最小 adapter 实验跑通，再把 OpenVLA 作为 frozen backbone 或强 baseline。

#### 3.3 为什么暂时不直接上宇树

宇树 Go2/G1/H1 的价值很大，但真实硬件把安全、实时控制、通信、摔倒保护、设备磨损全部引入了。第一篇可行实验应该先在 LIBERO/ManiSkill 做软件闭环，再考虑 Go2 的低风险验证。

## 4. 环境准备：从一台电脑到能跑第一个 demo

### 4.1 推荐硬件

| 配置项 | 最低可跑                              | 推荐                                      |
|-----|-----------------------------------|-----------------------------------------|
| 系统  | Ubuntu 22.04/24.04 或实验室 Linux 服务器 | Linux 原生优先；WSL 可用于离线训练，但仿真 GUI/渲染更容易出问题 |
| GPU | 8-12GB 显存可跑小模型离线实验                | 24GB 以上更舒服；ManiSkill/大批量 rollout 越大越好   |
| 内存  | 32GB                              | 64GB+                                   |
| 存储  | 100GB 空闲                          | 500GB+，机器人数据和视频很占空间                     |

### 4.2 推荐环境策略

不要把所有库装进一个环境。建议分两个环境：

| 环境名              | 用途                            | 原因                     |
|------------------|-------------------------------|------------------------|
| snnwam-libero    | LIBERO + 离线训练 + SNN adapter   | 主线环境，尽量稳定。             |
| snnwam-maniskill | ManiSkill/大规模仿真/后续 ES rollout | ManiSkill 依赖可能不同，单独隔离。 |

### 4.3 安装顺序原则

1. 先安装显卡驱动、CUDA/PyTorch，确认 `torch.cuda.is_available()` 为 `True`。
2. 再安装 LIBERO，先跑官方 demo，不要立刻改代码。
3. 再安装 SpikingJelly/snnTorch/Norse 中的一个 SNN 框架。第一版推荐 SpikingJelly 或 snnTorch，选一个即可。
4. 最后才安装 ManiSkill、OpenVLA、Unitree RL Lab 等扩展栈。

### 4.4 示意命令：LIBERO 主环境

注意：下列命令是操作骨架。LIBERO 官方 README 目前仍给出 Python 3.8.13、Torch 1.11/CUDA 11.3 等老版本组合；如果你在新服务器上安装，要以官方 issues/README 和实验室机器 CUDA 版本为准，不要机械照抄。

```
# 1) 建环境
conda create -n snnwam-libero python=3.8.13 -y
conda activate snnwam-libero

# 2) 克隆 LIBERO
git clone https://github.com/Lifelong-Robot-Learning/LIBERO.git
cd LIBERO

# 3) 按官方 README 安装依赖
pip install -r requirements.txt
# 按你的 CUDA 版本安装 torch；官方 README 示例是 torch==1.11.0+cu113
pip install -e .
```

```
# 4) 下载一个小数据集，先不要全量下载
python benchmark_scripts/download_libero_datasets.py --datasets libero_spatial

# 5) 跑官方 notebook 或最小环境 step demo，确认能 reset/step/渲染。
```

4.5 安装成功的检查点

| 检查点           | 命令/行为                                                         | 通过标准                |
|---------------|---------------------------------------------------------------|---------------------|
| Python 环境     | python --version                                              | 版本符合当前库要求           |
| PyTorch GPU   | python -c "import torch;<br>print(torch.cuda.is_available())" | 输出 True             |
| LIBERO import | python -c "import libero;<br>print('ok')"                     | 无报错                 |
| 数据下载          | 检查 datasets 目录                                                | libero_spatial 数据存在 |
| 环境 step       | 运行官方 OffScreenRenderEnv demo                                  | 能 reset 并 step 10 次 |



## 5. 第一阶段：只做离线机器人数据实验

### 5.1 数据长什么样

在图像分类里，一个样本是 image,label。在机器人 demonstration 里，一个样本通常是一整条 trajectory:

```
trajectory_i = {
    images:      [T, H, W, C],
    robot_state: [T, state_dim],
    actions:     [T, action_dim],
    language:    string,
    success:     bool or sparse reward
}
```

训练 temporal adapter 时，你不会一次喂完整 trajectory，而是切成时间窗口：

```
sample_t = {
    image_t,
    language,
    action_history = actions[t-k:t-1],
    optional_state_t,
    target_actions = actions[t:t+H],
    target_future_images = images[t+1:t+H]
}
```

### 5.2 第一版先不预测像素，预测视觉 latent

不要让模型直接生成未来 RGB 图像。第一版应冻结视觉编码器，把图像变成 latent，然后让 SNN/GRU 预测未来 latent。这样难度从“视频生成”降成“时序表征预测”。

```
z_t      = visual_encoder(image_t)
z_future = visual_encoder(image_{t+1:t+H})
model predicts: z_hat_future, action_hat_{t:t+H}
```

### 5.3 模型结构

```
image_t + language + action_history + optional_state_t
    ↓
frozen visual encoder + frozen text encoder
    ↓
projection / fusion MLP
    ↓
temporal adapter: MLP / GRU / SNN
    ↓
head 1: future action chunk
head 2: future visual latent chunk
    ↓
loss = action_loss + future_latent_loss + spike_loss
```

### 5.4 第一个 baseline : GRU adapter

不要一上来写 SNN。先写 GRU adapter，因为它是最直接的时序 baseline。只要 GRU 能跑通，SNN 的替换就有参照。

| 模块              | 输入                         | 输出                              |
|-----------------|----------------------------|---------------------------------|
| visual encoder  | image_t                    | z_img                           |
| text encoder    | instruction                | z_txt                           |
| history encoder | action_{t-k:t-1}           | z_hist                          |
| GRU adapter     | concat(z_img,z_txt,z_hist) | h_t -> action_hat, z_future_hat |

5.5 离线指标

| 指标                         | 计算方式                          | 说明          |
|----------------------------|-------------------------------|-------------|
| Action MSE                 | MSE(action_hat, action_demo)  | 最基本动作预测误差   |
| Future latent cosine error | 1 - cosine(z_hat, z_target)   | 最小 WAM 核心指标 |
| Multi-step degradation     | 随 horizon 增加的 latent error 曲线 | 看多步预测是否崩    |
| Spike rate                 | mean(spikes)                  | SNN 稀疏性指标   |
| Inference latency          | 单步推理时间                        | 后续机器人部署相关   |

## 6. 第二阶段：closed-loop 仿真到底怎么做

### 6.1 仿真不是“看视频”，而是一个可 step 的环境

仿真器的核心接口和强化学习环境类似：

```
obs = env.reset()
for t in range(max_steps):
    action = policy(obs, instruction)
    obs, reward, done, info = env.step(action)
    if done:
        break
success = info["success"] or reward == 1
```

这一步和离线测试的区别是：模型输出的 action 会真的改变环境。如果动作稍微错一点，下一帧图像就可能偏离 demonstration，因此 closed-loop success rate 通常比离线 action MSE 更重要。

### 6.2 LIBERO 中的

LIBERO 提供 OffScreenRenderEnv、任务套件、固定 initial states 和官方 evaluation 脚本。第一阶段你不需要写物理引擎，只需要调用它的环境。

```
from libero.libero import benchmark
from libero.libero.envs import OffScreenRenderEnv

benchmark_dict = benchmark.get_benchmark_dict()
task_suite = benchmark_dict["libero_spatial"]()
task = task_suite.get_task(0)
# 根据 task 的 bddl_file 构造 OffScreenRenderEnv
# env.reset(); env.set_init_state(...); env.step(action)
```

### 6.3 closed-loop evaluation 的最小版本

| 步骤 | 做什么                                     | 通过标准                |
|----|-----------------------------------------|---------------------|
| 1  | 加载训练好的 policy checkpoint                | 能 forward 输出 action |
| 2  | 选择 1 个 LIBERO task 和 5 个 initial states | 能 reset 到固定初始状态     |
| 3  | 每个 episode 最多执行 N 步                     | 不会中途报错              |
| 4  | 记录 reward/done/info                     | 能判断 success/failure |
| 5  | 保存视频或关键帧                                | 能人工检查失败模式           |
| 6  | 汇总 success rate                         | 输出 CSV              |

### 6.4 为什么离线误差好不代表闭环好

一个 action predictor 可能在 teacher-forcing 数据上误差很低，但一旦自己执行，进入了 demonstration 没覆盖的状态，就会连续犯错。这叫 distribution shift 或 compounding error。因此，具身论文必须报告 closed-loop success。

## 7. 第三阶段：把 GRU 换成 SNN

### 7.1 SNN 在这里不该做什么

- 不要一开始让 SNN 直接控制所有关节 torque。
- 不要声称 SNN 天然低功耗，除非你真的部署到神经形态硬件；先报告 spike rate / synaptic operation proxy。
- 不要只把 GRU 改名为 SNN，要说明它承担事件驱动时序记忆、短期动态和稀疏激活。

### 7.2 SNN 在这里应该做什么

SNN 是 temporal adapter：它接收 fused latent 和动作历史，内部保持膜电位状态，输出动作 chunk 和未来 latent。

```
for tau in sequence_window:
    input_tau = fused_feature_tau
    spike_tau, membrane_tau = snn_cell(input_tau, membrane_{tau-1})

h = readout(spike_sequence or membrane_sequence)
a_hat = action_head(h)
z_hat_future = future_head(h)
```

### 7.3 建议的 SNN 框架

| 框架           | 优点                                      | 建议                       |
|--------------|-----------------------------------------|--------------------------|
| SpikingJelly | PyTorch SNN 框架，中英文资料较多，支持 LIF/IF/PLIF 等 | 优先尝试                     |
| snnTorch     | PyTorch 风格清晰，教学材料多                      | 如果 SpikingJelly 版本冲突，可用它 |
| Norse        | PyTorch spiking components，接口较模块化       | 作为备选                     |

### 7.4 SNN loss

```
L_total = L_action + lambda_future * L_future + lambda_spike * L_spike

L_action = MSE(action_hat, action_demo)
L_future = 1 - cosine(z_hat_future, z_future)
L_spike = mean(spike_rate)
```

### 7.5 SNN 是否有价值，看这些实验

| 问题            | 实验                              | SNN 有价值的信号             |
|---------------|---------------------------------|------------------------|
| 是不是只是 RNN 换皮？ | 和 GRU/LSTM 比较                   | 同等 success 下更稀疏，或扰动下更稳 |
| 是否适合时序？       | action delay / frame drop       | 延迟或缺帧时退化更慢             |
| 是否适合 WAM？     | 多步 future latent prediction     | horizon 变长时误差增长更慢      |
| 是否能闭环完成任务？    | LIBERO rollout                  | success rate 接近或超过 GRU |
| 是否更像类脑？       | spike rate / state dynamics 可视化 | 出现稀疏、事件触发的动态响应         |



## 8. 第四阶段：EGGROLL-style ES 什么时候加入

### 8.1 先不要用 ES 从零训练

纯 ES 从零训练 SNN-WAM adapter 很可能慢，而且不一定比 Adam 稳。更合理的是混合路线：surrogate gradient warmup + EGGROLL-style ES post-training。

### 8.2 为什么 EGGROLL 和这个题有关

EGGROLL 的意义不是控制机器人，而是让 ES 更接近现代大模型/大模块训练范式：通过低秩扰动提高并行评估效率，并允许只要能做 batched inference、能定义 fitness，就可以优化系统参数。对 SNN-WAM 来说，这意味着你可以在可微监督训练之后，用 task success、动作平滑、安全惩罚、spike 稀疏性等系统级目标继续后训练。

### 8.3 三个层级的 ES 实验

| 层级                     | 优化对象                                        | fitness                                  | 目的                          |
|------------------------|---------------------------------------------|------------------------------------------|-----------------------------|
| 离线 ES sanity           | SNN adapter 小模块                             | - action_loss - future_loss - spike_cost | 验证 ES 代码能工作，但创新性有限          |
| 闭环 ES post-training    | adapter / action head / low-rank parameters | success - jerk - unsafe - spike_cost     | 真正体现黑箱优化价值                  |
| EGGROLL-style low-rank | 低秩扰动矩阵或 LoRA-like adapter                   | 同上                                       | 提高并行评估效率，接近老板分享 EGGROLL 的重点 |

### 8.4 最小 ES 伪代码

```
theta = adapter_parameters
for generation in range(G):
    eps = sample_low_rank_noise(population=N, rank=r)
    fitness = []
    for i in parallel_population:
        theta_i = theta + sigma * eps[i]
        score_i = evaluate(theta_i) # offline loss or closed-loop success
        fitness.append(score_i)
    theta = theta + alpha * weighted_sum(fitness, eps)
```

### 8.5 何时说明 EGGROLL 有必要

- 如果目标只是 action MSE，Adam/SG 已经足够，EGGROLL 没必要。
- 如果目标包含 closed-loop success、碰撞、安全、动作平滑、spike 稀疏等不可微或高噪声指标，EGGROLL-style ES 才有意义。
- 如果模型很小，普通 ES 也能跑；只有当 adapter 或 backbone 变大、并行评估多时，低秩扰动的意义才会显现。

## 9. 具体实验设计：第一篇可汇报版本

### 9.1 实验题目

暂定题目：Spiking World-Action Adapters for Language-Conditioned Robot Manipulation。

中文表述：面向语言条件机器人操作的脉冲 World-Action 适配器。

### 9.2 数据和任务

| 项目        | 第一版选择                                           | 原因                    |
|-----------|-------------------------------------------------|-----------------------|
| Benchmark | LIBERO-Spatial 或 LIBERO-Object                  | 任务数适中，语言条件明确          |
| 训练数据      | demonstration trajectories                      | 先做 imitation learning |
| 验证数据      | held-out trajectories / held-out initial states | 看离线泛化                 |
| 测试数据      | closed-loop rollout tasks                       | 看任务成功率                |

### 9.3 模型组

| 模型                | Temporal module        | 是否预测 future latent | 目的                        |
|-------------------|------------------------|--------------------|---------------------------|
| BC-MLP            | 无时序/简单 MLP             | 否/可选               | 最低 baseline               |
| BC-GRU            | GRU                    | 否                  | 强时序 baseline              |
| WAM-GRU           | GRU                    | 是                  | 判断 future prediction 是否有用 |
| WAM-SNN-LIF       | LIF SNN                | 是                  | 主方法最小版                    |
| WAM-SNN-ALIF/PLIF | 自适应阈值/参数化 LIF          | 是                  | SNN 变体                    |
| WAM-SNN-ES        | SNN + ES post-training | 是                  | 接 EGGROLL 训练范式            |

### 9.4 Baseline 和消融

| 对比                                 | 回答的问题                | 必须程度   |
|------------------------------------|----------------------|--------|
| MLP vs GRU                         | 时序记忆是否有用             | 必须     |
| GRU vs SNN                         | SNN 是否超过普通 RNN       | 必须     |
| with / without future latent loss  | WAM 目标是否提升闭环表现       | 必须     |
| with / without spike penalty       | 稀疏性和性能的 trade-off    | 必须     |
| surrogate-only vs ES post-training | EGGROLL-style 训练是否有用 | 第二阶段必须 |
| noise/delay/frame drop             | SNN 是否更鲁棒            | 必须     |
| 不同 horizon H                       | 多步预测是否稳定             | 必须     |

### 9.5 指标

| 指标 | 离线/闭环 | 意义 |
|----|-------|----|
|----|-------|----|

|                               |    |                |
|-------------------------------|----|----------------|
| Action MSE / L1               | 离线 | 动作预测基本能力       |
| Future latent cosine error    | 离线 | WAM 动态预测能力     |
| Multi-step latent error curve | 离线 | 多步 rollout 是否崩 |
| Closed-loop success rate      | 闭环 | 最重要任务完成指标      |
| Completion steps              | 闭环 | 完成速度           |
| Failure mode count            | 闭环 | 失败类型分析         |
| Spike rate / SynOps proxy     | 模型 | SNN 稀疏性        |
| Inference latency             | 模型 | 后续部署可能性        |

9.6 第一批图表

| 图表       | 内容                                                             |
|----------|----------------------------------------------------------------|
| Figure 1 | 模型结构图: frozen encoder + temporal adapter + future/action heads |
| Figure 2 | 训练/验证 loss 曲线                                                  |
| Figure 3 | Action MSE 对比柱状图                                               |
| Figure 4 | Future latent error 随 horizon 变化的曲线                            |
| Figure 5 | Closed-loop success rate 对比                                    |
| Figure 6 | Spike rate vs success rate trade-off                           |
| Figure 7 | noise/delay/frame drop 下鲁棒性曲线                                  |



## 10. 项目目录和代码组织

### 10.1 推荐目录

```
snn_wam_adapter/  
  README.md  
  environment.yml  
  configs/  
    libero_spatial_gru.yaml  
    libero_spatial_snn.yaml  
    libero_spatial_snn_es.yaml  
  src/  
    data/  
      libero_dataset.py  
      trajectory_window.py  
      transforms.py  
    models/  
      encoders.py  
      temporal_mlp.py  
      temporal_gru.py  
      temporal_snn.py  
      wam_adapter.py  
      heads.py  
    train/  
      train_offline.py  
      train_es_post.py  
    eval/  
      eval_offline.py  
      eval_rollout_libero.py  
      eval_robustness.py  
    utils/  
      metrics.py  
      logging.py  
      seed.py  
      checkpoint.py  
  scripts/  
    download_libero.sh  
    run_train_gru.sh  
    run_train_snn.sh  
    run_eval_offline.sh  
    run_eval_rollout.sh  
  results/  
    runs/  
    tables/  
    figures/  
  docs/  
    experiment_log.md  
    weekly_report.md  
    paper_outline.md
```

## 10.2 每次实验必须保存什么

| 文件               | 内容                           | 为什么重要    |
|------------------|------------------------------|----------|
| config.yaml      | 所有超参数和路径                     | 可复现      |
| metrics.csv      | 每个 epoch / task / seed 的指标   | 画图和统计    |
| checkpoint.pt    | 模型权重                         | 可复评估     |
| git_commit.txt   | 当前代码 commit                  | 避免结果不可追踪 |
| eval_rollout.csv | 每个 episode 的 success/failure | 闭环证据     |
| failure_videos/  | 失败 episode 视频或关键帧            | 分析失败模式   |
| notes.md         | 当天观察和异常                      | 方便给老板汇报  |

## 10.3 训练脚本的接口

```
python src/train/train_offline.py --config configs/libero_spatial_snn.yaml --dataset_root /path/to/LIBERO/datasets --output_dir results/runs/snn_wam_001

python src/eval/eval_offline.py --checkpoint results/runs/snn_wam_001/best.pt --split test

python src/eval/eval_rollout_libero.py --checkpoint results/runs/snn_wam_001/best.pt --suite libero_spatial --num_episodes 50
```

11. 12 周推进计划

| 周数     | 目标                          | 具体任务                                                               | 交付物                            |
|--------|-----------------------------|--------------------------------------------------------------------|--------------------------------|
| 第 1 周  | 具身入门和 LIBERO 跑通             | 安装环境；下载 libero_spatial；运行官方 demo；理解 observation/action/language 格式 | 环境安装记录；能 reset/step 的截图或日志     |
| 第 2 周  | 离线 dataloader               | 写 trajectory window dataset；可视化样本；检查 action 维度和图像 shape            | dataset sanity notebook；样本可视化图 |
| 第 3 周  | MLP/GRU baseline            | 训练 action prediction baseline；记录 train/val/test error              | 第一张 loss 曲线和 action MSE 表      |
| 第 4 周  | 加入 future latent prediction | 冻结视觉编码器；预测未来 latent；比较 with/without future loss                    | future latent error 曲线         |
| 第 5 周  | 闭环 evaluation               | 把 GRU policy 放回 LIBERO rollout；记录 success rate 和失败视频               | 第一张 success rate 表             |
| 第 6 周  | SNN adapter v1              | 实现 LIF SNN adapter；surrogate gradient 训练；先只做离线指标                   | SNN loss/spike rate 曲线         |
| 第 7 周  | SNN 闭环评估                    | 跑 LIBERO rollout；比较 GRU 和 SNN 的 success rate                       | 闭环对比表和失败案例                     |
| 第 8 周  | 鲁棒性实验                       | 加 observation noise、action delay、frame drop；比较退化曲线                 | 鲁棒性曲线                          |
| 第 9 周  | ES post-training v0         | 只在小模块上做离线 ES sanity；确认 fitness 能更新参数                               | ES 训练日志                        |
| 第 10 周 | 闭环 ES post-training         | 用 success/jerk/spike fitness 做少量 rollout 优化                        | surrogate-only vs ES 后训练表      |
| 第 11 周 | 整理结果和消融                     | 多 seed；补 ablation；统一图表风格                                           | results package                |
| 第 12 周 | 写论文雏形                       | 写 introduction、method、experiment；准备组会汇报                            | 8-10 页 draft 或 slides          |

## 12 周后应该能回答的问题

5. SNN adapter 是否能在离线 action prediction 和 future latent prediction 上跑通?
6. future latent prediction 是否提升 closed-loop success?
7. SNN 相比 GRU 是否在噪声、延迟、缺帧或长 horizon 下更稳?
8. SNN 的稀疏性是否和成功率形成可解释 trade-off?
9. EGGROLL-style ES 后训练是否能改善 task success 或系统级指标?

## 12. 第一周逐日清单：真正从零开始

| 天     | 任务                                                   | 通过标准                                       |
|-------|------------------------------------------------------|--------------------------------------------|
| Day 1 | 阅读 LIBERO README；记录安装要求；建 conda 环境                   | 能进入环境，python/torch 正常                      |
| Day 2 | 克隆 LIBERO，安装依赖                                       | import libero 不报错                          |
| Day 3 | 下载 libero_spatial 数据                                 | 数据目录存在，能看到 hdf5/npz/pkl 等数据文件              |
| Day 4 | 运行官方环境 step demo                                     | env.reset 和 env.step 能跑，能 offscreen render |
| Day 5 | 运行官方 BC/RNN 训练脚本或 notebook                           | loss 开始下降或能完成一个小训练 epoch                   |
| Day 6 | 阅读一个 trajectory，打印 image/state/action/language shape | 写出 data_inspect.ipynb                      |
| Day 7 | 写一页实验日志给老板                                           | 说明已跑通什么、卡在哪里、下一步是什么                        |

### 第一周不要做的事

- 不要改 LIBERO 源码，先跑通官方例子。
- 不要直接实现 SNN。
- 不要下载所有大数据集，先只下载 libero\_spatial 或 libero\_object。
- 不要上真实机器人。
- 不要先写论文引言；先拿到第一个可复现日志。

### 13. 风险清单和应对

| 风险                | 表现                                    | 应对                                                                    |
|-------------------|---------------------------------------|-----------------------------------------------------------------------|
| 安装失败              | MuJoCo/robosuite/CUDA 版本冲突            | 优先复现官方环境；记录完整报错；不要同时装太多库                                              |
| 离线指标好但闭环差         | action MSE 低，success rate 低           | 增加闭环评估；分析失败视频；考虑 action chunk、DAGGER 类思路或更强 policy                    |
| SNN 不如 GRU        | 所有指标都差                                | 检查是否任务不需要事件动态；增加 noise/delay/frame drop；调 spike penalty；不要强行声称 SNN 优势 |
| future latent 没帮助 | 加 WAM loss 后 success 不升反降             | 调整 loss 权重；只在 shared latent 上预测；检查视觉 encoder 是否合适                     |
| ES 太慢             | generation 很多但无提升                     | 只优化小 adapter；先 surrogate warmup；减少 population；先离线 fitness             |
| 实验不可复现            | 同一 config 结果差异大                       | 固定 seed；保存 commit；每个表至少 3 seeds；保存失败案例                                |
| 论文主张过大            | 实验只是小任务，却声称 embodied foundation model | 主张收敛为 spiking world-action adapter，不要声称完整基础模型                         |

#### 最重要的自我约束

这条线很容易变成“什么都想做”：SNN、VLA、WAM、EGGROLL、Unitree、人工生命。第一篇必须压缩成一个问题：SNN temporal adapter 是否能在语言条件操作中更好地建模未来状态和动作，并能通过混合训练优化闭环表现？

## 14. 组会汇报模板

### 14.1 每周 5 分钟汇报

本周目标：

- 例如：跑通 LIBERO-Spatial 的数据读取和环境 step。

完成内容：

- 已安装环境；下载数据；跑通 OffScreenRenderEnv；打印 action/image shape。

证据：

- 日志路径：results/runs/...
- 图表：data\_shape.png / loss\_curve.png

发现的问题：

- 例如：官方依赖 torch 版本较旧，新服务器 CUDA 版本需要调整。

下周计划：

- 写 trajectory window dataset；训练 MLP/GRU baseline。

### 14.2 第一次正式组会汇报结构

10. 问题：为什么 VLA 还不够，需要 WAM/未来状态建模。
11. 方法：冻结视觉语言编码器，比较 MLP/GRU/SNN temporal adapter。
12. 实验：LIBERO-Spatial/Object，离线 prediction + closed-loop rollout。
13. 指标：action error、future latent error、success rate、spike rate、鲁棒性。
14. 后续：ES/EGGROLL-style 后训练优化不可微系统级 fitness。

### 14.3 一句话版本

我想先把类脑和具身的结合落在一个可执行切口上：不直接训练完整 SNN-VLA/WAM，而是在现有视觉语言表示后加入一个 SNN world-action adapter，让它同时预测未来 latent 和动作，并用 LIBERO 的离线数据与闭环仿真验证它是否比 GRU/MLP 更适合时序扰动和系统级目标。

# 15. 论文雏形

## 15.1 可能题目

- Spiking World-Action Adapters for Language-Conditioned Robot Manipulation
- Training Spiking World-Action Adapters with Hybrid Gradient and Evolutionary Optimization
- Event-Driven Latent Dynamics for Vision-Language-Action Robot Policies

## 15.2 贡献写法

- 15. 提出 Spiking World-Action Adapter：把 SNN 放在冻结视觉语言 backbone 后，承担短期 latent dynamics、未来状态预测和动作调制。
- 16. 提出离线 prediction + closed-loop rollout 的评估协议，区分 action imitation、future latent modeling 和真实任务成功率。
- 17. 系统比较 MLP、GRU、Transformer/SNN adapters，并在噪声、延迟、缺帧和长 horizon 下评估时序鲁棒性。
- 18. 探索 surrogate-gradient warmup + low-rank evolutionary post-training，用于优化 task success、动作平滑和 spike 稀疏等系统级目标。

## 15.3 审稿人可能问什么

| 问题                          | 回答方式                                                    |
|-----------------------------|---------------------------------------------------------|
| 为什么不用 GRU/LSTM?             | 必须公平比较；如果 SNN 赢不了 success，就看鲁棒性、稀疏性和延迟扰动。               |
| 为什么这是 WAM，不只是 VLA?          | 因为模型显式预测 action-conditioned future latent，并报告多步预测和闭环效果。 |
| 为什么需要 ES?                   | 只在不可微/系统级 fitness 上使用 ES，不用它替代所有梯度训练。                   |
| 为什么不用 OpenVLA 直接 fine-tune? | 第一阶段是方法验证；后续可把 adapter 接到 OpenVLA/OFT 上。                |
| 硬件在哪里?                      | 第一篇可以没有硬件，但如果实验室有 Unitree，可作为小规模后续验证。                   |



## 16. 附录 A：最小配置文件示例

```
experiment:
  name: libero_spatial_snn_wam
  seed: 0

data:
  suite: libero_spatial
  history_len: 4
  action_horizon: 8
  future_horizon: 8
  image_size: 128

model:
  visual_encoder: frozen_resnet_or_clip
  text_encoder: frozen_clip_text_or_t5_small
  temporal_adapter: snn_lif
  hidden_dim: 256
  spike_penalty: 0.001

training:
  batch_size: 64
  epochs: 50
  optimizer: adamw
  lr: 0.0003
  lambda_future: 0.5
  lambda_spike: 0.001

eval:
  offline_metrics: [action_mse, future_latent_cosine, spike_rate]
  rollout:
    num_episodes: 50
    max_steps: 500
    record_video: true
```

## 17. 附录 B：结果表模板

| Model        | Action MSE ↓ | Future Error ↓ | Success ↑ | Spike Rate ↓ | Latency ↓ | Notes        |
|--------------|--------------|----------------|-----------|--------------|-----------|--------------|
| MLP          | -            | -              | -         | -            | -         | 最低 baseline  |
| GRU          | -            | -              | -         | -            | -         | 强时序 baseline |
| GRU + Future | -            | -              | -         | -            | -         | WAM 目标消融     |
| SNN-LIF      | -            | -              | -         | -            | -         | 主方法 v1       |
| SNN-ALIF     | -            | -              | -         | -            | -         | SNN 变体       |
| SNN + ES     | -            | -              | -         | -            | -         | 后训练          |



## 18. 附录 C：参考资料和官方入口

以下资料用于确定本计划的软件选择和阶段划分。安装时以官方 README / docs 为准。

| 资料                     | 地址                                                                                                                                              | 用途                                               |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| LIBERO GitHub          | <a href="https://github.com/Lifelong-Robot-Learning/LIBERO">https://github.com/Lifelong-Robot-Learning/LIBERO</a>                               | 语言条件机器人操作 benchmark；官方 README 提供安装、数据下载、训练和评估脚本。 |
| LIBERO paper / project | <a href="https://libero-project.github.io/main.html">https://libero-project.github.io/main.html</a>                                             | lifelong robot learning 和知识迁移 benchmark。         |
| ManiSkill 3 docs       | <a href="https://maniskill.readthedocs.io/en/latest/user_guide/index.html">https://maniskill.readthedocs.io/en/latest/user_guide/index.html</a> | GPU 并行机器人仿真和 manipulation framework。             |
| LeRobot docs           | <a href="https://huggingface.co/docs/lerobot/index">https://huggingface.co/docs/lerobot/index</a>                                               | PyTorch 机器人学习工具、数据集、模型与真实机器人入口。                  |
| OpenVLA GitHub         | <a href="https://github.com/openvla/openvla">https://github.com/openvla/openvla</a>                                                             | 开源 VLA 代码和 fine-tuning / evaluation 指南。          |
| OpenVLA paper          | <a href="https://arxiv.org/abs/2406.09246">https://arxiv.org/abs/2406.09246</a>                                                                 | 7B VLA，训练于约 970k 真实机器人示范。                        |
| SpikingJelly GitHub    | <a href="https://github.com/fangwei123456/spikingjelly">https://github.com/fangwei123456/spikingjelly</a>                                       | 基于 PyTorch 的 SNN 框架，中英文文档。                       |
| snnTorch docs          | <a href="https://snntorch.readthedocs.io/">https://snntorch.readthedocs.io/</a>                                                                 | PyTorch SNN 训练工具。                                |
| Norse docs             | <a href="https://norse.github.io/norse/">https://norse.github.io/norse/</a>                                                                     | PyTorch spiking components。                      |
| EGGROLL project        | <a href="https://eshyperscale.github.io/">https://eshyperscale.github.io/</a>                                                                   | Evolution Strategies at the Hyperscale 项目页。      |
| EGGROLL paper          | <a href="https://arxiv.org/abs/2511.16652">https://arxiv.org/abs/2511.16652</a>                                                                 | 低秩扰动 ES，面向大规模黑箱优化。                               |
| Unitree RL Lab         | <a href="https://github.com/unitreerobotics/unitree_rl_lab">https://github.com/unitreerobotics/unitree_rl_lab</a>                               | 基于 Isaac Lab 的 Unitree Go2/H1/G1-29dof 强化学习环境。   |

## 19. 最后一页：从零开始的正确心态

你以前做图像分类，重点是“数据到标签”的静态映射；现在做具身，重点变成“动作改变世界，世界反馈下一步观察”的闭环系统。入门时不要被 VLA/WAM/SNN/EGGROLL 这些词吓住。第一步仍然是工程上朴素的：把数据读出来，把 baseline 跑起来，把指标记下来，把模型放回环境执行。

当你能稳定回答三个问题时，就真正入门了：第一，我的模型离线预测得怎样？第二，它闭环执行能完成任务吗？第三，在噪声、延迟、缺帧或更长 horizon 下，它为什么成功或失败？

这条线与长期的人工生命式具身智能路线的连接点在于：你不是只做动作预测，而是在训练一个有内部时序状态、能预测未来世界变化、能用系统级目标调节自身行为的具身模块。SNN 和 EGGROLL 的价值，都应该围绕这个核心问题展开。